Week 3 - CloudFormation core (parameters, mappings, conditions, outputs, intrinsic functions, pseudo parameters)

DOP-C02 Domain 2 (Configuration Management and IaC). You must **read a template and spot what's wrong**, pick the **right function or section** for a requirement, and know the **rules and limits** that make a stack fail.

Part 1 - Template anatomy

```
AWSTemplateFormatVersion: "2010-09-09"     # optional, only valid value
Description: Web tier for the orders app   # optional, plain text
Metadata:                                  # optional (console parameter groups, cfn-init config)
  AWS::CloudFormation::Interface: {...}
Parameters: {...}                          # inputs at create / update time
Rules: {...}                               # validate parameter combinations before anything runs
Mappings: {...}                            # static lookup tables
Conditions: {...}                          # true / false switches
Transform: [AWS::Serverless-2016-10-31]    # macros: SAM, LanguageExtensions, Include, custom
Resources: {...}                           # THE ONLY REQUIRED SECTION
Outputs: {...}                             # values to show / export
```

Section	Required?	Intrinsic functions?	Purpose
`Resources`	**Yes**, at least one	Yes	What to create
`Parameters`	No	**No**, literal values	Inputs
`Mappings`	No	**No**, literal values only	Static lookup tables
`Conditions`	No	Condition functions plus `Ref` to **parameters**, `FindInMap`, pseudo parameters	When to create things
`Outputs`	No	Yes	Return or export values
`Rules`	No	Rule functions only	Validate parameters
`Transform`	No	-	Run macros before processing

- **Logical ID** = the name in the template (`WebServer`). **Physical ID** = the real name or ID (`i-0abc...`). **Stack** = one deployed template. YAML or JSON.
- **Template size: 51,200 bytes** passed directly (`--template-body`); **1 MB** from **S3** (`--template-url`).
- **Quotas per template: 500 resources**, **200 parameters**, **200 mappings**, **200 outputs**.

Part 2 - Parameters

```
Parameters:
  Environment:
    Type: String
    AllowedValues: [dev, test, prod]
    Default: dev
  InstanceType:
    Type: String
    Default: t3.micro
    AllowedPattern: "^t3\\..*"
    ConstraintDescription: must be a t3 instance type
  DesiredCount:
    Type: Number
    MinValue: 1
    MaxValue: 10
  VpcId:
    Type: AWS::EC2::VPC::Id                # console shows a dropdown of real VPCs
  Subnets:
    Type: List<AWS::EC2::Subnet::Id>
  LatestAmi:
    Type: AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>
    Default: /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64
  DbPassword:
    Type: String
    NoEcho: true
    MinLength: 12
```

Parameter types

Type	Notes
`String`, `Number`	Basic. Numbers are still passed as strings to resources
`List<Number>`, `CommaDelimitedList`	Lists. Use `!Select` / `!Join`
AWS-specific types (`AWS::EC2::KeyPair::KeyName`, `AWS::EC2::VPC::Id`, `AWS::EC2::Subnet::Id`, `List<AWS::EC2::Subnet::Id>`, `AWS::EC2::SecurityGroup::Id`, `AWS::EC2::Image::Id`, `AWS::Route53::HostedZone::Id`)	CloudFormation **checks the value exists in the account and Region** before creating anything. The console shows a dropdown
SSM parameter types (`AWS::SSM::Parameter::Value<String>`, `...<AWS::EC2::Image::Id>`, `...<List<String>>`)	The value is the **name** of a Parameter Store parameter, read **at every create and update**. Perfect for the **latest AMI**. **SecureString not supported**

Constraints, NoEcho, updates

- Constraints: `AllowedValues`, `AllowedPattern` (regex), `MinLength` / `MaxLength`, `MinValue` / `MaxValue`, `ConstraintDescription`, `Default`, `Description`.
- `NoEcho: true` masks the value as `****` in the console, CLI and API describe calls. It does **not** protect the value if you put it in **Outputs**, **Metadata**, or a visible property. **Better for secrets: dynamic references** (Part 7).
- On update, **Use existing value** (`UsePreviousValue=true`) keeps a parameter. Only resources that **reference** a changed parameter change.

Rules section

```
Rules:
  ProdNeedsLargeInstance:
    RuleCondition: !Equals [!Ref Environment, prod]
    Assertions:
      - Assert: !Contains [[m5.large, m5.xlarge], !Ref InstanceType]
        AssertDescription: prod must use m5.large or m5.xlarge
```

- Rules run **before** any resource is created or changed; the operation fails immediately with your message.

- Rule functions: `Fn::Contains`, `Fn::EachMemberEquals`, `Fn::EachMemberIn`, `Fn::RefAll`, `Fn::ValueOf`, `Fn::ValueOfAll`, plus `And` / `Or` / `Not` / `Equals`.
- `AWS::CloudFormation::Interface` in Metadata (`ParameterGroups`, `ParameterLabels`) orders and labels parameters in the console only.

Part 3 - Mappings

```
Mappings:
  EnvSettings:
    dev:
      InstanceType: t3.micro
      MinSize: "1"
    prod:
      InstanceType: m5.large
      MinSize: "3"
  RegionAmi:                              # older pattern (SSM parameter is better today)
    us-east-1: { Ami: ami-0abc111 }
    eu-west-1: { Ami: ami-0def222 }
Resources:
  Server:
    Type: AWS::EC2::Instance
    Properties:
      InstanceType: !FindInMap [EnvSettings, !Ref Environment, InstanceType]
      ImageId: !FindInMap [RegionAmi, !Ref "AWS::Region", Ami]
```

- **Exactly two levels:** map name, top-level key, second-level key, value.
- **Values must be literals.** No `Ref`, no functions, no parameters inside Mappings.
- The **lookup keys** can come from `Ref` to parameters or pseudo parameters (`AWS::Region`).
- A `FindInMap` for a missing key **fails the stack**. A `DefaultValue` fallback needs the `AWS::LanguageExtensions` transform.
- **Use for** values known in advance that change rarely. **Not for** frequently changing values like the **latest AMI** (use an SSM parameter type or dynamic reference), or values a user chooses (use a parameter).

Part 4 - Conditions

```
Conditions:
  IsProd: !Equals [!Ref Environment, prod]
  IsNotProd: !Not [!Condition IsProd]
  CreateReplica: !And
    - !Condition IsProd
    - !Equals [!Ref "AWS::Region", us-east-1]
  HasKeyPair: !Not [!Equals [!Ref KeyName, ""]]

Resources:
  ReadReplica:
    Type: AWS::RDS::DBInstance
    Condition: CreateReplica                # whole resource created only if true
    Properties: {...}
  Server:
    Type: AWS::EC2::Instance
    Properties:
      InstanceType: !If [IsProd, m5.large, t3.micro]
      KeyName: !If [HasKeyPair, !Ref KeyName, !Ref "AWS::NoValue"]   # drop the property

Outputs:
  ReplicaEndpoint:
    Condition: CreateReplica
    Value: !GetAtt ReadReplica.Endpoint.Address
```

Function	Use
`Fn::Equals`	Compare two values
`Fn::And` / `Fn::Or`	Combine **2 to 10** conditions
`Fn::Not`	Negate
`Condition` (inside conditions)	Reuse another condition
`Fn::If`	Choose a value **inside properties or outputs**: `!If [CondName, ifTrue, ifFalse]`
`AWS::NoValue`	With `!If`, **removes the property** completely

- Conditions can use **parameters, mappings, pseudo parameters and other conditions**. They **can't reference resources** (no `Ref` to a resource, no `GetAtt`), because they are evaluated before resources exist.
- The `Condition:` attribute goes on **resources and outputs**; property-level choices use `Fn::If`.
- If a resource is skipped by its condition, **resources that `Ref` or `GetAtt` it need the same condition** (or `!If`), otherwise validation fails.
- On update, if a condition becomes **false**, the resource is **deleted**. Protect databases with a `DeletionPolicy` (next topic).

Part 5 - Outputs, Exports and ImportValue

```
# network stack
Outputs:
  VpcId:
    Description: VPC for app stacks
    Value: !Ref Vpc
    Export:
      Name: !Sub "${AWS::StackName}-VpcId"    # e.g. network-prod-VpcId

# app stack
Resources:
  AppSg:
    Type: AWS::EC2::SecurityGroup
    Properties:
      VpcId: !ImportValue network-prod-VpcId
      GroupDescription: app
```

- **Outputs** show values in the console / `describe-stacks`, feed **CodePipeline output variables**, and pass **nested stack** outputs to the parent (`!GetAtt NestedStack.Outputs.Name`).
- **Export names are unique per account and per Region.**
- **Same Region only:** you can't import across Regions or accounts. Use SSM Parameter Store, StackSets or a custom resource instead.
- You **can't delete the exporting stack**, or **change or remove an export**, while another stack imports it ("Export ... cannot be updated as it is in use by ..."). Remove the imports first.
- `ImportValue` can't use a `Ref` to a resource in the importing stack; it can use `!Sub` with parameters to build the name.
- Up to **200 outputs**. **Don't output secrets**: even `NoEcho` parameters show if you output them.
- **Exports** suit **separate lifecycles** (a shared network stack used by many app stacks). **Nested stacks** suit **reusable components deployed together**.

Part 6 - Intrinsic functions

Function	Returns	Example
`Ref`	Parameter: its **value**. Resource: usually its **physical ID** (instance ID, bucket name, SG ID; for some resources the ARN)	`!Ref MyBucket` gives the bucket name
`Fn::GetAtt`	A **resource attribute**	`!GetAtt MyBucket.Arn`, `!GetAtt Alb.DNSName`, `!GetAtt Db.Endpoint.Address`
`Fn::Sub`	A string with variables replaced	`!Sub "arn:${AWS::Partition}:s3:::${MyBucket}/*"`
`Fn::Join`	Values joined with a delimiter	`!Join [",", [a, b, c]]`
`Fn::Split`	A list from a string	`!Split [",", !Ref SubnetCsv]`
`Fn::Select`	One item from a list by index	`!Select [0, !GetAZs ""]`
`Fn::GetAZs`	The AZs in a Region (`""` = current Region)	`!GetAZs !Ref "AWS::Region"`
`Fn::FindInMap`	A value from Mappings	`!FindInMap [Map, Key1, Key2]`
`Fn::If` / `Equals` / `And` / `Or` / `Not`	Condition logic	See Part 4
`Fn::ImportValue`	An exported value	`!ImportValue network-VpcId`
`Fn::Base64`	Base64 encoding (for **EC2** `UserData`)	`Fn::Base64` wrapping a `!Sub` block
`Fn::Cidr`	A list of CIDR blocks	`!Cidr [!GetAtt Vpc.CidrBlock, 4, 8]` (4 subnets, /24)
`Fn::Transform`	Calls a macro on part of the template	`AWS::Include` snippet from S3
`Fn::Length`, `Fn::ToJsonString`, `Fn::ForEach`, FindInMap `DefaultValue`	Newer helpers	Need `Transform: AWS::LanguageExtensions`

Fn::Sub in detail

```
# 1. Variables: parameters, resources (Ref), attributes, pseudo parameters
BucketPolicyArn: !Sub "arn:${AWS::Partition}:s3:::${LogBucket}/AWSLogs/${AWS::AccountId}/*"
QueueName: !Sub "${MyQueue.QueueName}"           # ${Resource.Attribute} = GetAtt

# 2. Literal ${} (for example in bash): escape with ${!...}
Command: !Sub "echo ${!HOME}"                     # outputs: echo ${HOME}

# 3. Map form: define your own variables
Url: !Sub
  - "https://${Domain}/api"
  - Domain: !ImportValue shared-ApiDomain
# 4. UserData pattern
UserData:
  Fn::Base64: !Sub |
    #!/bin/bash
    echo "env=${Environment}" > /etc/app.conf
    /opt/aws/bin/cfn-signal -e $? --stack ${AWS::StackName} --resource Asg --region ${AWS::Region}
```

YAML short-form gotcha

- Some short forms can't be nested **directly**: `!Base64 !Sub "..."` on one line is invalid. Use a long form outside (`Fn::Base64: !Sub ...`) or inside (`!Base64 {"Fn::Sub": "..."}`).
- `!GetAtt` **takes a literal** `Resource.Attribute`. You **can't** use `!Ref` to choose the resource name inside it.

Implicit dependencies

- `Ref` **or** `GetAtt` **creates an automatic dependency**: the referenced resource is created first and deleted last.
- Use `DependsOn` only when there is no reference but order matters (for example an instance that needs the NAT route or the VPC gateway attachment first).

Part 7 - Dynamic references (secrets and parameters at deploy time)

Syntax	Reads	Notes
`{{resolve:ssm:/app/db/host}}` (or `:3` for version 3)	Parameter Store **String / StringList**	Resolved at create and update. Without a version, latest
`{{resolve:ssm-secure:/app/db/password:1}}`	Parameter Store **SecureString**	**Only specific resource properties** (for example RDS `MasterUserPassword`, IAM user passwords)
`{{resolve:secretsmanager:prod/db:SecretString:password}}`	**Secrets Manager** (whole secret or a **JSON key**, optional version stage / ID)	**Preferred for secrets.** Works with rotation; value never appears in the template or stack events

```
Database:
  Type: AWS::RDS::DBInstance
  Properties:
    MasterUsername: "{{resolve:secretsmanager:prod/db:SecretString:username}}"
    MasterUserPassword: "{{resolve:secretsmanager:prod/db:SecretString:password}}"
    # or let RDS manage it:  ManageMasterUserPassword: true
```

- Resolved **at stack create or update**. A later secret change doesn't update the resource **until the next stack update** that touches it, and isn't detected as drift.
- Create secrets in the template with `AWS::SecretsManager::Secret` + `GenerateSecretString`, attached with `AWS::SecretsManager::SecretTargetAttachment`; or RDS `ManageMasterUserPassword: true`.
- **SSM parameter types** appear as **stack parameters** (visible, overridable). `{{resolve:ssm:...}}` is written **inline** in properties.

Part 8 - Pseudo parameters

Pseudo parameter	Value	Typical use
`AWS::AccountId`	`111122223333`	ARNs, unique bucket names
`AWS::Region`	`us-east-1`	Region-specific values, `FindInMap`, ARNs
`AWS::StackName`	`orders-prod`	Naming and tags, export names, `cfn-signal`
`AWS::StackId`	The full stack ARN	Unique IDs
`AWS::Partition`	`aws`, `aws-cn`, `aws-us-gov`	**Portable ARNs**: `arn:${AWS::Partition}:...`
`AWS::URLSuffix`	`amazonaws.com` (or `amazonaws.com.cn`)	Portable service URLs
`AWS::NotificationARNs`	The stack's SNS topic ARNs	Pass notifications along
`AWS::NoValue`	Removes a property	With `Fn::If`

- **Never hard-code** account ID, Region, partition or AZ names. Use pseudo parameters and `!GetAZs`, so the **same template works in every account and Region** (needed for StackSets).

Part 9 - Resource naming and update behaviour

Behaviour	Meaning
No interruption	Changed in place
Some interruption	For example an EC2 reboot
Replacement	New physical resource, references updated, **old one deleted** (data loss risk)

- **Preview with a change set**, which shows `Replacement: True` (next topic).
- **Custom names** (`BucketName`, `TableName`, `RoleName`): CloudFormation **can't replace** a resource with a fixed name, so replacement updates **fail**, and the template can't be deployed twice in one account and Region. **Let CloudFormation generate names**, or include `${AWS::StackName}` / `${AWS::Region}`.
- **Capabilities:** IAM resources with custom names need `CAPABILITY_NAMED_IAM`, other IAM resources `CAPABILITY_IAM`, macros and nested transforms `CAPABILITY_AUTO_EXPAND`.
- **Validation:** `aws cloudformation validate-template` checks **syntax only**. Use **cfn-lint** for property checks and **cfn-guard** for policy rules.
- **Packaging:** `aws cloudformation package` uploads local code (Lambda zips, nested templates) to **S3** and rewrites paths; `aws cloudformation deploy` creates or updates **through a change set**.

Part 10 - Exam traps

Trap	Truth
Use `Ref` inside Mappings to set a value from a parameter	Mapping values are **literals only**
A condition checks a resource or uses `GetAtt`	Conditions use **parameters, mappings, pseudo parameters, other conditions** only
Remove an optional property when a parameter is empty	`!If [HasValue, !Ref Param, !Ref "AWS::NoValue"]`
Import a VPC ID from a stack in another Region	**ImportValue is same-Region only.** Use SSM parameters, StackSets or a custom resource
Update the exporting stack to rename an export	**Fails while imported.** Remove imports first
NoEcho keeps the password safe everywhere	Only masks describe calls. **Outputs and Metadata leak it.** Use **Secrets Manager dynamic references**
SSM SecureString parameter type for passwords	**Not supported.** Use `{{resolve:ssm-secure:...}}` (limited) or `{{resolve:secretsmanager:...}}`
Always the latest AMI without editing the template	`AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>` with the public AMI parameter
Hard-coded AMI map per Region	Works but goes stale. **SSM parameter** is the modern answer
`Ref` returns the ARN for every resource	Mostly the **physical ID**. Use `GetAtt ...Arn`
`validate-template` confirms the stack will deploy	**Syntax only.** Use cfn-lint and change sets
Template is 70 KB, pass with `--template-body`	Direct limit **51,200 bytes**. Use **S3** (`--template-url`, 1 MB) or `package`
`arn:aws:` in China or GovCloud	Use `${AWS::Partition}`
Hard-code `us-east-1a`	`!Select [0, !GetAZs ""]`
Bash variables in UserData break with `!Sub`	Escape as `${!VAR}`
Fixed `BucketName`, update needs replacement	**Fails.** Avoid custom names or include stack name / Region
Stack fails with a capabilities error	`CAPABILITY_IAM` / `CAPABILITY_NAMED_IAM` / `CAPABILITY_AUTO_EXPAND`
Prod must use large instances, validated before deploy	**Rules** section with assertions
FindInMap with a fallback value	`AWS::LanguageExtensions` (`DefaultValue`)
Loop to create 5 similar resources	`Fn::ForEach` (LanguageExtensions), or CDK / modules

Part 11 - Hands-on lab (about 60 to 90 minutes, free tier friendly)

Lab 1 - Parameters, Mappings, Conditions, Outputs in one template

Save as `web.yaml`:

```
AWSTemplateFormatVersion: "2010-09-09"
Description: Lab - core CloudFormation features
Parameters:
  Environment:
    Type: String
    AllowedValues: [dev, prod]
    Default: dev
  LatestAmi:
    Type: AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>
    Default: /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64
  KeyName:
    Type: String
    Default: ""
Mappings:
  EnvSettings:
    dev:  { InstanceType: t3.micro }
    prod: { InstanceType: t3.small }
Conditions:
  IsProd: !Equals [!Ref Environment, prod]
  HasKey: !Not [!Equals [!Ref KeyName, ""]]
Resources:
  WebSg:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: !Sub "web ${Environment} in ${AWS::Region}"
      SecurityGroupIngress:
        - { IpProtocol: tcp, FromPort: 80, ToPort: 80, CidrIp: 0.0.0.0/0 }
  Web:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: !Ref LatestAmi
      InstanceType: !FindInMap [EnvSettings, !Ref Environment, InstanceType]
      KeyName: !If [HasKey, !Ref KeyName, !Ref "AWS::NoValue"]
      SecurityGroupIds: [!GetAtt WebSg.GroupId]
      AvailabilityZone: !Select [0, !GetAZs ""]
      UserData:
        Fn::Base64: !Sub |
          #!/bin/bash
          dnf install -y nginx
          echo "<h1>${Environment} from ${AWS::StackName} - home is ${!HOME}</h1>" >
    /usr/share/nginx/html/index.html
          systemctl enable --now nginx
      Tags:
        - { Key: Name, Value: !Sub "${AWS::StackName}-web" }
  ProdAlarmTopic:
    Type: AWS::SNS::Topic
    Condition: IsProd
Outputs:
  Url:
    Value: !Sub "http://${Web.PublicDnsName}"
  SgId:
    Value: !GetAtt WebSg.GroupId
    Export:
      Name: !Sub "${AWS::StackName}-WebSgId"
  TopicArn:
    Condition: IsProd
    Value: !Ref ProdAlarmTopic
```

1. Run `cfn-lint web.yaml` (`pip install cfn-lint`) and fix any warnings.
2. `aws cloudformation deploy --template-file web.yaml --stack-name lab-core`.
3. Open the **Url** output: the page shows `dev`, the stack name, and the **literal** `${HOME}` (the escape worked).
4. Confirm there is **no SNS topic** (condition false) and the **TopicArn output is missing**.

Lab 2 - Flip the condition and read the change set

1. `aws cloudformation deploy ... --parameter-overrides Environment=prod --no-execute-changeset`.
2. Open the change set: **InstanceType** modified, **SNS topic Add**. Check whether the instance is **replaced or modified** (instance type change = stop/start, some interruption).
3. Execute the change set.

Lab 3 - Cross-stack import and the export lock

1. Create stack `lab-app` with a security group rule using `!ImportValue lab-core-WebSgId` as `SourceSecurityGroupId`.
2. Try to **delete** `lab-core`: it **fails** because the export is in use.
3. Delete `lab-app` first, then `lab-core`.

Lab 4 - Dynamic reference to Secrets Manager

1. Create secret `lab/db` with JSON `{"username":"admin","password":"Lab-Pass-12345"}`.
2. Add an `AWS::SSM::Parameter` (or a tag) using `{{resolve:secretsmanager:lab/db:SecretString:username}}`. Use the **username**, not the password, because tags and parameters are visible.
3. Deploy and confirm the value. The template and events show only the reference.

Lab 5 - Rules and validation

1. Add a `Rules` assertion: prod must use t3.small.
2. Change the mapping so prod becomes t3.micro and deploy with `Environment=prod`: it fails **before any change** with your message.
3. Run `validate-template` on a template with `InstanceTyp` misspelled: it **passes** (syntax only), but cfn-lint and the deployment catch it.

Clean up

- Delete stacks `lab-app` and `lab-core`, and secret `lab/db` (recovery window, or `--force-delete-without-recovery` for a lab).

Part 12 - Check yourself

Questions

1. Which template section is the only required one?
2. Can you use `!Ref Environment` inside a Mappings value? As a lookup key in `FindInMap`?
3. How do you make an EC2 `KeyName` optional, left out when a parameter is empty?
4. A team must always launch the newest Amazon Linux AMI without editing templates. What parameter type?
5. What is the difference between `Ref` and `Fn::GetAtt` for an S3 bucket?
6. UserData uses `$HOME` inside `!Sub` and the value is wrong. How do you fix it?
7. A stack in eu-west-1 must use a VPC ID exported in us-east-1. Will `ImportValue` work? Alternatives?
8. You try to change an export's value and the update fails. Why?
9. Why is `NoEcho` not enough for a database password, and what is better?
10. Which pseudo parameter makes ARNs work in China and GovCloud Regions?
11. Can a condition check whether a resource was created by the stack?
12. `validate-template` succeeded but deployment failed with "Encountered unsupported property". Why?
13. A template is 80 KB. How do you deploy it?
14. An update fails because a resource with `BucketName: my-app-logs` needs replacement. Cause and better practice?
15. How do you stop a stack from deploying prod with a small instance type, before any resource is touched?

Answers

1. `Resources`.
2. **No** for the value (literals only). **Yes** for the lookup key: `!FindInMap [Map, !Ref Environment, Key]`.
3. Condition `HasKey: !Not [!Equals [!Ref KeyName, ""]]`, then `KeyName: !If [HasKey, !Ref KeyName, !Ref "AWS::NoValue"]`.
4. `AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>` with the public `/aws/service/ami-amazon-linux-latest/...` parameter as default.
5. `Ref` returns the **bucket name**; `GetAtt Bucket.Arn` returns the **ARN** (also `DomainName`, `RegionalDomainName`).
6. Escape as `${!HOME}`.
7. **No**, exports are **same-Region only**. Write the value to **SSM Parameter Store** in the target Region (StackSets or pipeline), use a **custom resource**, or pass it as a parameter.
8. **Another stack imports it.** Remove the imports first.
9. NoEcho only masks describe calls; the value can leak via **Outputs, Metadata or properties**, and someone must type it. Use `{{resolve:secretsmanager:...}}` or RDS `ManageMasterUserPassword`.
10. `AWS::Partition`.
11. **No.** Conditions can't reference resources.
12. `validate-template` checks **syntax only**. Use **cfn-lint**.
13. Upload to **S3** and use `--template-url` (1 MB), or `package` + `deploy`. Direct limit is 51,200 bytes.
14. A **fixed custom name** blocks the replacement. Don't set custom names, or include `${AWS::StackName}` / `${AWS::Region}`; protect data with a DeletionPolicy.
15. A `Rules` section with a `RuleCondition` (prod) and an **assertion** on the instance type.

Quick memory summary

- **Resources** is the only required section. Parameters and Mappings hold **literals**; Conditions use **parameters, mappings, pseudo parameters, other conditions** (never resources).
- **Parameters:** AWS-specific types (validated), **SSM parameter types** (latest AMI; no SecureString), constraints, NoEcho (masks describe only), **Rules** for combinations.
- **Mappings:** two levels, `FindInMap`, DefaultValue needs LanguageExtensions.
- **Conditions:** Equals / And / Or / Not / If; `AWS::NoValue` removes properties; false on update deletes the resource.
- **Outputs / Exports:** unique names per account and Region, **same Region only**, locked while imported.
- **Functions:** Ref (value / physical ID), **GetAtt** (attributes, ARN), **Sub** (`${!x}` escape), Join, Split, Select, GetAZs, Base64 (UserData), Cidr, ImportValue, Transform.
- **Dynamic references:** `resolve:ssm`, `resolve:ssm-secure` (limited), `resolve:secretsmanager` (preferred).
- **Pseudo parameters:** AccountId, Region, StackName, StackId, **Partition**, URLSuffix, NotificationARNs, NoValue.
- **Limits:** 51,200 bytes direct / 1 MB S3; 500 resources; 200 parameters, mappings, outputs. **Custom names block replacement.** `validate-template` = syntax only.